

HAI-DAR APP — COMPLETE HANDOVER

Current-source backup and developer / AI continuation documentation

Prepared from the supplied 62-file source export. No product features or UI were redesigned during this handover.

1. PROJECT OVERVIEW

HAI-DAR is a mobile-first B2B wholesale ordering application for Indian hand tools. The current source is designed for wholesalers, distributors and dealers to browse a factory catalogue, select product variants, respect/display minimum order quantities, build a bulk cart, send enquiries, place order requests and track order status. A protected admin control room manages products, order/enquiry queues and business settings.

Primary business flow: Customer opens catalogue → searches/categories → opens product → selects variant and quantity → adds to bulk cart or uses WhatsApp enquiry → signs in with development OTP when required → submits order/enquiry → data is stored in MongoDB → admin sees the queue and updates status → customer sees the updated status in Order/Enquiry detail.

Current UI direction is industrial red/black/white with wholesale-first copy and a single Expo Router entry screen.

2. TECHNOLOGY STACK

Area	Current Implementation
Frontend	Expo 57.0.24, Expo Router 57.0.22, React Native 0.86.3, TypeScript
State/UI	React state/hooks; most application state and screens are orchestrated in `frontend/app/index.tsx`
Backend	FastAPI 0.110.1 + Uvicorn
Database driver	Motor 3.3.1 / PyMongo 4.6.3
Database	MongoDB; database name comes from `DB_NAME`
Authentication	JWT HS256 via PyJWT; 14-day expiry; admin passwords via passlib bcrypt
Frontend auth storage	Expo SecureStore on native; AsyncStorage-backed secure helpers on web
API architecture	REST-style JSON API under `/api`
Frontend package manager	Yarn 1.22.22
Deployment	Emergent-managed preview/deploy is documented; self-host via Uvicorn/reverse proxy is also documented
Native builds	Expo/EAS path is documented
Unavailable/unknown	Exact production host, production MongoDB instance, live secrets and live deployment values are NOT FOUND / intentionally excluded

3. COMPLETE PROJECT STRUCTURE

Path	Purpose
backend/server.py	Complete FastAPI app, models, seed/bootstrap, auth dependencies, all API routes
backend/requirements.txt	Pinned Python dependency set
backend/tests/backend_test.py	Backend regression/integration tests
backend/tests/test_admin_auth.py	Admin authentication/protected-route tests
backend/.env.template	Backend environment variable names/placeholders
frontend/app/index.tsx	Main Expo application: screens, state, auth flow, cart, admin control room
frontend/app/_layout.tsx	Root Stack, ErrorBoundary, QueryClientProvider, keyboard provider
frontend/app/+html.tsx	Web HTML shell/reset
frontend/src/haidar-api.ts	Typed API client, money formatter, WhatsApp URL helper
frontend/src/theme.ts	Light industrial theme tokens and styles helper

Path	Purpose
frontend/src/utis/storage/*	Native/web persistence abstraction
frontend/src/components/error-boundary.tsx	Render error boundary
frontend/src/query-client.ts	TanStack Query client
frontend/assets/*	Expo icons/splash/font/image assets
frontend/scripts/*	Expo/project guard/reset tooling
frontend/constants/testIds/*	Test IDs for auth/navigation
frontend/package.json	Frontend scripts/dependencies
frontend/app.json	Expo app configuration
memory/PRD.md	Original product requirements/context
design_guidelines.json	UI/design token specification
HANDOVER.md	Existing long-form handover generated by the source project

4. DATABASE

There is no migration framework. MongoDB is schemaless and the backend performs idempotent startup bootstrap. It creates a unique index on `admins.email`, seeds settings if absent, seeds four starter products if the product collection is empty, and seeds one admin if no admin exists.

Collection	Important fields / notes
products	id, name, category, sku, description, specifications[], packaging, delivery, image_data[], variants[], featured, is_new, active, badge, created_at
settings	Single business document: id='business', whatsapp, phone, email, address, delivery_info, announcement
customers	id, mobile, name, business_name, city, state, created_at
admins	id, email, password_hash, created_at; unique email index
orders	id, customer_id, items[], customer_name, business_name, mobile, total, status, tracking, created_at
enquiries	id, customer_id, customer_name, business_name, mobile, whatsapp, city, state, product, required_quantity, message, items[], status, sales_response, sales_notes, created_at
otp_codes	Legacy/dev flow: mobile, code='dev', created_at; current verification does not compare this record

Relationships: `orders.customer\_id` and `enquiries.customer\_id` reference customer IDs; `orders.items[].product\_id` references products; variants are embedded in products; categories are derived from product documents. No separate categories or variants collection exists.

5. AUTHENTICATION

Customer

- Frontend asks for a 10-digit mobile number.
- POST /api/auth/request-otp is a development-only mock and returns dev_code 1234.
- POST /api/auth/verify-otp accepts any 4-digit numeric code in the current source; it creates/upserts a customer by mobile and returns a customer JWT.
- JWT payload contains sub, role='customer' and exp. Expiry is 14 days.
- Native token storage uses SecureStore; web secure storage falls back to AsyncStorage.
- Customer profile data is stored separately in general storage and restored on boot.

Admin

- POST /api/auth/admin-login uses email + password against the MongoDB admins collection.
- Admin password is bcrypt-verified; an unknown email still runs a dummy bcrypt verification.

- Admin JWT has role='admin'.
- Admin token uses a separate storage key from the customer token, allowing buyer and admin sessions to coexist on one device.
- All /api/admin/* routes depend on admin_identity and return 401/403 when appropriate.

6. CUSTOMER FEATURES

Feature	Current source implementation
Home	Catalogue hero, announcement, categories, featured products, trust cards and CTAs
Categories	Derived from active product categories; filter chips in Products screen
Search/filter	Search name/SKU/category/variant label/SKU; category filter; sort by featured/name/price
Product detail	Description, specs, packaging, delivery, variant selection, price, MOQ and quantity
Variant selection	Embedded variants with label/SKU/weight/MOQ/price/stock
Cart	React state only; add/remove/quantity; no server-side cart
MOQ	Quantity starts at selected variant MOQ and is displayed client-side; backend order endpoint does not enforce MOQ
Enquiry	Form posts to /api/enquiries; current frontend requires customer login
Orders	Customer order list + tappable customer detail with six-stage timeline
Profile	Customer info, login/logout, enquiry access, admin access entry
WhatsApp	Deep-link generated from settings.whatsapp using wa.me

7. ADMIN FEATURES

Feature	Current implementation
Authentication	Email/password login; protected admin token
Products	List/create/update/delete via /api/admin/products
Categories	No separate category CRUD; category is a product string field
Variants	Edited as embedded product.variants[]
Price/MOQ/stock	Editable inside variants; stock is display-only
Images	image_data[] field exists, but current admin UI has no upload pipeline
Featured/new/badge	Product fields and admin editing support
Orders	Admin queue reads /api/admin/orders; PATCH updates status/tracking
Order detail	NOT IMPLEMENTED in admin UI or backend detail route
Enquiries	Admin queue reads /api/admin/enquiries; PATCH updates status/sales_notes
Customers	No dedicated admin customer-management endpoint/UI found
Settings	WhatsApp, phone, email, address, delivery info, announcement

8. ORDER FLOW

Current order flow: Customer → Cart → POST /api/orders → MongoDB `orders` collection → Admin Orders queue → PATCH /api/admin/orders/{id} updates `status` and `tracking` → Customer GET /api/orders/{id} shows detail/timeline.

Status values represented by the current documentation/UI: Enquiry Received → Order Confirmed → Processing → Packed → Dispatched → Delivered.

Important implementation detail: the backend accepts the submitted `total`, item quantities and prices from the client. It does not currently recalculate product pricing, enforce MOQ, or reserve/decrement stock.

9. ENQUIRY FLOW

Current source flow: Customer signs in → fills name, business, mobile, WhatsApp, city, state, product, quantity and message → POST /api/enquiries → MongoDB `enquiries` → Admin Enquiries queue → admin PATCH can update status and sales_notes → customer GET /api/enquiries/{id} displays detail and sales_response when present.

The older handover/PRD says guest enquiry is allowed, but the current backend dependency requires JWT and the frontend also blocks submission when logged out. This is documented as a source-level discrepancy rather than treated as a working feature.

WhatsApp CTA is separate from database enquiry submission and opens a `wa.me` deep link using the configured sales number.

10. API DOCUMENTATION

Method/path	Auth	Request	Response / errors
GET /api/	None	—	Health JSON
GET /api/settings	None	—	Business settings
PATCH /api/admin/settings	Admin	JSON partial settings	Updated settings; 401/403 if unauthenticated/wrong role
GET /api/products	None	search, category, sort, page, limit	items/categories/page/has_more
GET /api/products/{id}	None	id	Product; 404 if unavailable
GET/POST/PATCH/DELETE /api/admin/products	Admin	Product JSON / partial JSON / id	CRUD result; auth and not-found errors
POST /api/auth/request-otp	None	mobile query	Dev OTP metadata; 400 for short mobile
POST /api/auth/verify-otp	None	mobile, code query	Customer token + user; 400/401 validation errors
POST /api/auth/admin-login	None	email,password query	Admin token + role; 401 generic auth error
POST /api/enquiries	JWT currently required	Enquiry JSON	Created enquiry; 401 if no JWT
GET /api/enquiries	JWT	—	Caller-scoped enquiries
GET /api/enquiries/{id}	JWT	id	Caller-owned enquiry; 404 otherwise
PATCH /api/admin/enquiries/{id}	Admin	status, sales_notes query	Updated enquiry; 404 if missing
POST /api/orders	JWT	items, customer fields, total JSON	Created order
GET /api/orders	JWT	—	Caller-scoped orders
GET /api/orders/{id}	JWT	id	Caller-owned order; 404 otherwise
GET /api/admin/orders	Admin	—	All orders
PATCH /api/admin/orders/{id}	Admin	status, tracking query	Updated order; 404 if missing

11. ENVIRONMENT VARIABLES

Variable	Purpose	Secret?
MONGO_URL	MongoDB connection string	YES — provide locally/deployment secret
DB_NAME	MongoDB database name	No, but environment-specific
JWT_SECRET	HS256 JWT signing secret	YES
ADMIN_EMAIL	Bootstrap admin email	Sensitive operational value
ADMIN_PASSWORD	Bootstrap admin password on first startup	YES
EXPO_PUBLIC_BACKEND_URL	Frontend API base URL	No
EXPO_TUNNEL_SUBDOMAIN	Emergent-managed dev value	Environment-specific
EXPO_PACKAGER_HOSTNAME	Emergent-managed dev value	Environment-specific
EXPO_PACKAGER_PROXY_URL	Emergent-managed dev value	Environment-specific
EXPO_USE_FAST_RESOLVER	Emergent-managed dev value	Environment-specific

Variable	Purpose	Secret?
METRO_CACHE_ROOT	Emergent-managed dev value	Environment-specific
ADMIN_TEST_PASSWORD	Sanitized test-suite placeholder introduced in final backup	YES

No actual secret values are included in this handover. Use placeholders such as `YOUR_DATABASE_URL_HERE`, `YOUR_JWT_SECRET_HERE` and `YOUR_ADMIN_PASSWORD_HERE`.

12. DEPLOYMENT / LOCAL SETUP

Backend: Python 3.11, MongoDB 6/7, virtualenv, ``pip install -r backend/requirements.txt``, copy `backend/.env.template` to `backend/.env`, fill secrets, then run ``uvicorn server:app --host 0.0.0.0 --port 8001 --reload`` from backend.

Frontend: Node 20.x is documented, Yarn 1.x is pinned. Copy `frontend/.env.template` to `frontend/.env`, set ``EXPO_PUBLIC_BACKEND_URL``, run ``yarn install``, then ``yarn start``. Web/mobile/simulator options are available through Expo.

Production: Emergent-managed deploy is documented through Publish → Deploy your app. Self-host deployment uses MongoDB + HTTPS backend + reverse proxy preserving `/api`, then Expo/EAS build with `EXPO_PUBLIC_BACKEND_URL` set before build.

Database migration: no migration framework. Startup performs idempotent seeding. This is important for another AI: do not invent migration files that are not present.

13. CURRENT KNOWN ISSUES — VERIFIED FROM SOURCE

Issue	Evidence / impact
Admin order detail gap	No GET <code>/api/admin/orders/{id}</code> ; AdminRecords renders summary rows without <code>onPress/detail</code> handler.
Admin enquiry detail gap	No GET <code>/api/admin/enquiries/{id}</code> ; admin queue is summary-only.
Enquiry guest-mode mismatch	Current backend requires <code>current_identity</code> for POST <code>/api/enquiries</code> ; frontend also requires login, contrary to older PRD wording.
OTP is mock	Any 4-digit numeric code is accepted; no real SMS provider.
OTP record not actually validated	<code>request-otp</code> writes <code>otp_codes</code> , but <code>verify-otp</code> only validates code format and does not compare stored code.
No server-side MOQ enforcement	POST <code>/api/orders</code> accepts client quantity; client starts at MOQ but backend does not reject below MOQ.
Stock is display-only	No stock decrement/reservation in order creation.
Cart is not persisted	React state only; device/app lifecycle can lose it.
No image upload	<code>image_data</code> exists but current UI does not upload images.
Admin queues not paginated	Admin GET queues return all documents.
No admin customer management	No admin customer CRUD/list endpoint found.
Runtime test limitation	Full pytest not run here because <code>pytest-xdist</code> was unavailable in the inspection runtime; frontend build not run without installed dependencies/Yarn.

14. BUSINESS RULES

- B2B / wholesale positioning is factory-direct; copy avoids unsupported superlatives.
- MOQ belongs to each variant, not the whole product.
- Prices are displayed in INR with no decimals using the ``money()`` helper.
- Order status uses a six-stage pipeline.
- Admin can edit product, variant, business settings, order status/tracking and enquiry status/notes.
- Admin and customer sessions are separate.
- WhatsApp number is configured in ``settings.whatsapp`` without plus/spaces in the intended format.
- All timestamps written by backend use UTC ISO strings.

15. FUTURE INTEGRATION POINTS

- Real SMS OTP provider: replace mock request/verify logic and persist OTPs with expiry.
- Payment gateway: order total is already present; payment state is not yet modeled.
- Shipping API: integrate around order status/tracking.
- Invoice generation: attach generation/download flow to completed/dispatch stages.
- Notifications: email/push can trigger from order/enquiry status changes.
- WhatsApp API: current implementation is a deep link, not an official API integration.
- Production image storage: populate image_data[] with public object-storage URLs.

16. TROUBLESHOOTING

Symptom	Diagnosis / action
Frontend says request failed	Check EXPO_PUBLIC_BACKEND_URL and that backend is reachable; inspect '/api/' health endpoint.
401 on customer orders	Customer JWT missing/expired; clear stored customer token and sign in again.
403 on admin route	Token is a customer token or JWT role is not admin.
Admin login fails after fresh DB	Check ADMIN_EMAIL/ADMIN_PASSWORD and startup seeding logs.
Products not visible	Check MongoDB connection/DB_NAME and backend startup seed; inspect GET /api/products.
WhatsApp opens wrong number	Check settings.whatsapp; normalize to digits only.
Admin order appears but cannot open	Expected current limitation: admin detail route/UI is not implemented.
Tests cannot run	Install backend requirements including pytest-xdist; configure EXPO_BACKEND_URL and valid test admin environment.
Web auth persistence differs	Web secure storage falls back to AsyncStorage by design; native uses SecureStore.

17. HOW ANOTHER AI SHOULD CONTINUE THIS PROJECT

The next AI must first inspect the existing source code, not rebuild from scratch. Preserve the existing Expo/FastAPI/MongoDB architecture and current UI. Preserve database field names and API paths unless a requested change genuinely requires migration. Fix bugs surgically. Ask before major architectural changes. Never invent missing business data or credentials. Never expose secrets. When changing an endpoint, inspect both the backend route and every frontend caller before editing. When changing auth, preserve the separate customer/admin token namespaces and role gates.

Recommended inspection order: `HANDOVER.md` → `backend/server.py` → `frontend/app/index.tsx` → `frontend/src/haidar-api.ts` → storage/theme files → tests. Treat source code as authoritative when it conflicts with older PRD text.

18. CURRENT PROJECT CHECKLIST

Feature	Status	Important files	Notes
Customer OTP login	IMPLEMENTED (DEV ONLY)	backend/server.py; frontend/app/index.tsx	Any 4-digit numeric code passes
Admin authentication	IMPLEMENTED	backend/server.py; frontend/app/index.tsx	DB-backed bcrypt + JWT
Home/catalogue	IMPLEMENTED	frontend/app/index.tsx; backend/server.py	Public catalogue
Search/filter/sort	IMPLEMENTED	frontend/app/index.tsx; backend/server.py	Search + category + sort
Products CRUD	IMPLEMENTED	backend/server.py; frontend/app/index.tsx	Admin protected
Variants/MOQ/price	IMPLEMENTED	Product model + ProductDetail	MOQ client-side
Cart	IMPLEMENTED	frontend/app/index.tsx	In-memory React state

Feature	Status	Important files	Notes
Order creation	IMPLEMENTED	backend/server.py; frontend/app/index.tsx	Server accepts client total
Customer order detail	IMPLEMENTED	backend/server.py; frontend/app/index.tsx	Six-stage timeline
Admin order queue	IMPLEMENTED	backend/server.py; frontend/app/index.tsx	List only
Admin order detail	NOT IMPLEMENTED	No admin detail route/UI	Known gap
Enquiry submission	IMPLEMENTED WITH LOGIN	backend/server.py; frontend/app/index.tsx	Current source requires JWT
Admin enquiry queue	IMPLEMENTED	backend/server.py; frontend/app/index.tsx	List only
Admin enquiry detail/edit response	PARTIAL	backend/server.py	Status + sales_notes only; no admin detail UI
WhatsApp deep links	IMPLEMENTED	frontend/src/haidar-api.ts	Uses settings.whatsapp
Product image upload	NOT IMPLEMENTED	image_data only	Placeholder field
Admin customer management	NOT IMPLEMENTED	No route/UI	No dedicated feature
Payments	NOT IMPLEMENTED	None	Future integration
Shipping/invoices	NOT IMPLEMENTED	None	Future integration
Push notifications	NOT IMPLEMENTED	None	Future integration

19. VERIFICATION SUMMARY

- The supplied ZIP is a real ZIP containing 62 source/documentation files, not screenshots.
- The final source backup preserves the supplied folder structure and source files, with only the hard-coded test/admin password redacted to comply with the no-secrets requirement.
- backend/server.py and both backend test modules pass Python syntax compilation.
- All API routes listed in this document were located directly in backend/server.py.
- Customer token persistence and separate admin token persistence were located directly in frontend/app/index.tsx and the storage abstraction.
- The full pytest suite was not claimed as executed because the inspection runtime lacked pytest-xdist; frontend build was not claimed because dependencies were not installed.

Final status: HANDOVER PACKAGE READY. This package documents the current source as inspected; it does not claim live production health beyond the source-level verification described above.